HORRAZON AI

Enterprise AI for Business Intelligence & Automation

AI Engineering Challenge

Round II
Technical Assessment

Document Type	Engineering Assessment
Position	AI / ML Engineer Intern (LLM & Agentic AI)
Client Simulation	NovaCart Global (Fictional Enterprise Client)
Assignment Duration	36 Hours from time of release
Round/Stage	Round II
Version	1.0
Issued By	Horrazon AI – AI & Engineering Team

1. Welcome

Congratulations on reaching the assignment stage of Horrazon AI's hiring process for the AI / ML Engineer Intern (LLM & Agentic AI) role.

This is not a competitive-programming test, a chatbot exercise, or a prompt-engineering quiz. It is a compact simulation of the kind of work you would actually do at Horrazon AI: taking an ambiguous business problem, designing a reasonable architecture, building a working system against a deadline, and being able to explain and defend every decision you made.

You are free to use AI coding assistants (Claude, ChatGPT, Copilot, Cursor, Gemini, etc.), public documentation, and open-source libraries. What we are evaluating is your judgment — not whether you typed every line yourself.

> **Why an internship candidate gets an "enterprise" brief**
>
> The scenario below is intentionally large — it mirrors a real Horrazon AI client engagement. You are NOT expected to fully build NovaCart Global's platform in 36 hours. You are expected to scope a sensible slice of it, build that slice well, and clearly document what you deliberately left out and why. Section 6 defines the minimum slice we expect.

2. About Horrazon AI

Horrazon AI is an AI-first technology company building an enterprise intelligence platform that turns fragmented business data into explainable, evidence-backed answers and recommended actions — not just dashboards.

The platform draws on several disciplines that this internship track will expose you to directly:

- Retrieval-Augmented Generation (RAG)
- Multi-agent AI systems and tool-using agents
- Large language models and prompt/context engineering
- Knowledge graphs and enterprise search
- Workflow automation and decision intelligence

Horrazon AI does not hire on coding ability alone. We look for engineers who can understand a business problem, design a scalable architecture around it, make and defend trade-offs, and ship something that works — even under a tight deadline.

3. The Role: AI / ML Engineer Intern — LLM & Agentic AI

As an intern on this track, you will work alongside the AI Engineering team on retrieval, reasoning, and agentic systems for Horrazon AI's platform. Interns are given real scope — not busywork — under senior engineer supervision. Typical responsibilities you'd grow into include:

- Building and tuning RAG pipelines (chunking, embeddings, retrieval, reranking)
- Designing and implementing tool-using / multi-step agents
- Wiring LLM orchestration layers (LangChain, LangGraph, or custom)
- Adding evaluation, observability, and guardrails around AI systems
- Collaborating with backend engineers on APIs that expose AI capabilities

This assessment is your first (simulated) assignment on that team.

4. The Scenario: NovaCart Global

Horrazon AI has been engaged by a fictional client, NovaCart Global, a fast-growing multinational e-commerce company:

Metric	Value
Customers	1.2 million
Monthly orders	350,000+
Employees	6,000+
Warehouses	5 (New York, Chicago, Dallas, Berlin, Singapore)
Suppliers	30
Countries	12
Annual revenue	~$480 million

NovaCart grew through acquisitions, and every acquired company brought its own tech stack — SAP, Oracle ERP, Dynamics, Salesforce, HubSpot, Zendesk, Power BI, Confluence, Jira, and more. As a result, no single business question ("Why did refunds increase last month?", "Which supplier causes the most delays?") can be answered without manually digging through spreadsheets, emails, PDFs, and dashboards across several departments — a process that today takes hours or days.

NovaCart has hired Horrazon AI to build a unified AI layer: a system where executives and managers can ask natural-language business questions and get evidence-backed,

explainable answers that reason across multiple disconnected data sources, rather than a keyword search over one system.

5. What the Full Platform Would Eventually Do

For context (not all of this is in scope for your 36 hours — see Section 6), the full production platform is expected to:

- Ingest CSV, Excel, JSON, TXT, Markdown, DOCX, PDF, scanned PDF, images, emails, and logs
- Support semantic, keyword, hybrid, and metadata-filtered search
- Reason across multiple sources rather than retrieve-and-paste (e.g. refund rate → complaints → warehouse logs → shipment delays → supplier quality → timeline → explanation)
- Attach evidence to every claim and separate facts from assumptions and recommendations
- Maintain multi-turn conversational memory ("compare it with February", "show supporting evidence")
- Generate structured, stakeholder-ready reports

Scale target for the production system (not required in this assessment): 10M documents, 100M embeddings, 1,000 concurrent users.

6. Your Assignment: Scoped Deliverable for This Assessment

Given the 36-hour window, build a focused, working prototype that demonstrates the core reasoning loop above on a small, self-contained synthetic dataset — not the entire NovaCart estate. Specifically:

6.1 Minimum required scope

- Create or use a small synthetic dataset (10–30 documents is enough) spanning at least 3 of the following: orders, refunds, support tickets, supplier contracts/quality reports, warehouse/shipment logs, marketing campaigns, internal emails. Deliberately include a couple of inconsistencies (an outdated policy, a duplicate record, a missing field) — you don't need OCR noise or thousands of records.
- Build an ingestion + chunking + embedding + indexing pipeline for that dataset (any vector store: pgvector, Qdrant, Chroma, etc.)
- Build a retrieval layer supporting at least semantic search, with metadata filtering on at least one dimension (e.g. department, date, or source type)
- Build a reasoning/agentic layer that can answer a multi-hop business question by pulling from more than one document type and chaining that evidence into an answer (a single tool-using agent or a simple LangGraph/LangChain pipeline is sufficient — it does not need to be a full multi-agent system)

- Every answer must cite which documents/records it used and flag when evidence is missing or the answer is uncertain
- Expose the system through a REST API (chat/query endpoint at minimum) with at least one other supporting endpoint (e.g. document upload or health check)
- Basic auth is acceptable (a single hardcoded or seeded user is fine) — the point is to show you know where auth belongs, not to build enterprise SSO

6.2 Encouraged if time allows (not required)

- A minimal front-end or CLI to interact with the system
- Multi-turn conversational memory ("compare it with...")
- A short structured report generation endpoint
- Basic evaluation harness (a handful of test questions with expected evidence sources)
- Structured logging / simple observability

6.3 Explicitly out of scope

- Model fine-tuning or training custom LLMs
- Kubernetes, distributed infra, enterprise SSO, billing, mobile/voice interfaces
- Reproducing the full NovaCart dataset described in Section 4 — a believable, small, synthetic slice is enough

> **Use good judgment**
>
> If you can only get part of Section 6.1 fully working, ship the part you completed cleanly, and use your README to explain exactly what's missing and how you'd complete it with more time. A small system that works and is well explained beats a large system that half-works and is undocumented.

7. Technical Expectations

7.1 Free choice of stack

Choose any language, framework, LLM provider, embedding model, database, vector store, and orchestration framework you're comfortable with (Python, Node.js, Go, Java, Rust; LangChain, LangGraph, Haystack, LlamaIndex, or custom; OpenAI, Anthropic, Google, OpenRouter, or local models). Every choice should be justifiable in the interview — the reasoning matters more than which option you pick.

7.2 Engineering hygiene

- Clean, modular structure (don't put everything in one file)
- Reasonable error handling around file processing, embedding calls, and LLM calls

- No hardcoded secrets — use environment variables with a sample .env.example
- At least a few unit or integration tests

7.3 Deployment

- The project must run via Docker (Dockerfile at minimum; docker-compose is a plus if you have a DB + app)
- Clear, reproducible setup instructions in the README

8. Deliverables

Submit all of the following:

Deliverable	Notes
GitHub repository	Public or invite the reviewer; commit history should reflect real, incremental work
README	Overview, architecture, tech stack, setup, environment variables, known limitations, future work
Architecture diagram	One diagram is enough — image or ASCII/mermaid is fine
Docker configuration	Dockerfile (+ compose if applicable)
Design decisions note	1–2 pages: why this stack, why this chunking/retrieval strategy, what you'd change for production
Demo video	5–10 minutes, walking through the running system and explaining decisions — not just a UI tour

9. Timeline

> **36 Hours**
>
> The countdown starts the moment this assignment is released to you. Submit before the deadline via the link/method provided by your recruiting contact. Late submissions may not be reviewed unless a delay was agreed in advance — if something comes up, tell us as early as possible rather than after the deadline.

Suggested pacing (a guide, not a rule):

Window	Focus
Hours 0–4	Read the brief fully, decide your stack and scope, sketch the architecture, set up the repo and Docker skeleton
Hours 4–20	Build ingestion, retrieval, and the reasoning/agent layer against your synthetic dataset
Hours 20–28	Wire the REST API, add error handling, write tests, tighten evidence/citation behavior
Hours 28–34	Write the README and design-decisions note, finish Docker setup, record the demo video
Hours 34–36	Final review, push, submit

10. Allowed Resources & Academic Integrity

You are encouraged to use official documentation, Stack Overflow, research papers, open-source libraries, and AI coding assistants (Claude, ChatGPT, Copilot, Cursor, Gemini, Perplexity). Using AI tools is expected, not penalized.

You must not, however:

- Copy an existing GitHub repository and present it as your own work
- Submit another person's work
- Use closed-source proprietary code without permission
- Misrepresent AI-generated code as something you don't understand — you will be asked to explain and modify it live in the interview

11. How This Will Be Evaluated

We weigh engineering judgment more heavily than feature count. Broadly, we look at:

Area	What we're looking for
AI Engineering	Sensible chunking, embeddings, prompt design, and retrieval quality; resistance to hallucination
Agentic Design	Whether the reasoning/agent layer genuinely chains evidence across sources rather than answering from one document

Software Engineering	Structure, modularity, readability, error handling, tests
System Design	Sensible separation of concerns and a design that could plausibly grow, even if it doesn't need to today
Backend & API Design	Clean REST endpoints, basic auth awareness, async handling where it matters
Communication	Can you clearly explain and defend what you built, and what you'd do differently with more time?

A short follow-up interview will ask you to walk through your architecture, defend a few decisions, and make a small live modification (e.g. swap a component, add a filter, handle a larger upload). We're checking that you understand your own system — not testing recall.

12. Frequently Asked Questions

- **Can I use AI assistants?** Yes, throughout.
- **Can I use open-source libraries?** Yes.
- **Do I need to implement every "encouraged" item in 6.2?** No — they're bonus signal, not requirements.
- **Can I deploy this locally only?** Yes, a local Docker setup is sufficient. A live deployment is a nice-to-have, not required.
- **What if I run out of time?** Submit what you have, working, with a clear README describing what's incomplete and how you'd finish it.

13. Final Note

We are not expecting a finished enterprise platform in 36 hours — nobody builds NovaCart's full intelligence layer in that time, and we'd be suspicious of a submission that claimed to. We're looking for a well-scoped, well-reasoned, working slice of it, built and explained the way you'd operate on a real Horrazon AI engineering team.

Good luck — we're looking forward to seeing what you build.

— Horrazon AI & Engineering Team